Agentic Self-Learning Retrieval-Augmented Code Generation System for Java

NLP Internship Technical Report

Beshoy Emad Fawzy

March 12, 2026

Contents

Abstract

This report presents an adaptive, self-learning Retrieval-Augmented Generation (RAG) system designed for Java code generation.

The system combines semantic retrieval, multi-agent orchestration using LangGraph, execution-based validation, conversational memory, and continual learning to create an intelligent programming assistant.

Unlike traditional code generation systems that rely solely on Large Language Models (LLMs), the proposed architecture grounds generation in real code examples retrieved from a vector database and verifies generated programs through actual Java compilation.

The system operates locally on consumer-grade hardware and is deployed as a scalable API using FastAPI and Docker with GPU acceleration.

1 Introduction

Large Language Models (LLMs) have recently demonstrated strong capabilities in program synthesis. However, they suffer from several limitations:

- Hallucinated code
- Lack of grounding in real repositories
- No runtime verification
- Static knowledge boundaries

Retrieval-Augmented Generation (RAG) addresses these limitations by combining:

- semantic search
- contextual grounding
- language model reasoning

This work extends classical RAG by introducing an **Agentic RAG architecture**, where specialized agents collaborate to generate, validate, and improve code.

The system focuses specifically on Java code generation.

2 Hardware Environment

The system was designed to run efficiently on consumer hardware:

- CPU: AMD Ryzen 7 5800H
- RAM: 40GB
- GPU: NVIDIA RTX 3050 (4GB VRAM)

To fit within GPU memory constraints, lightweight models and optimized inference strategies were used.

3 Dataset Construction

The retrieval system relies on a dataset derived from publicly available Java code examples.

Each dataset entry follows the structure:

```
{
  "id": sample_id,
  "prompt": natural_language_description,
  "code": java_function
}
```

The dataset is embedded into a vector database using a sentence embedding model. Let:

$$\mathcal{D} = \{(p_i, s_i)\}_{i=1}^{N}$$

where

- p_i represents a natural language prompt
- s_i represents the corresponding Java implementation

4 Mathematical Formulation of RAG

4.1 Embedding Function

An embedding model transforms prompts into dense vectors.

$$E : \mathcal{P} \to \mathbb{R}^d$$

$$v_q = E(q)$$

$$v_i = E(p_i)$$

4.2 Similarity Search

Cosine similarity is used for retrieval.

$$sim(q, p_i) = \frac{v_q \cdot v_i}{\|v_q\|_2 \|v_i\|_2}$$

The top-k most similar prompts are selected:

$$\mathcal{R}_k(q) = \arg\operatorname*{topk}_i \, sim(q, p_i)$$

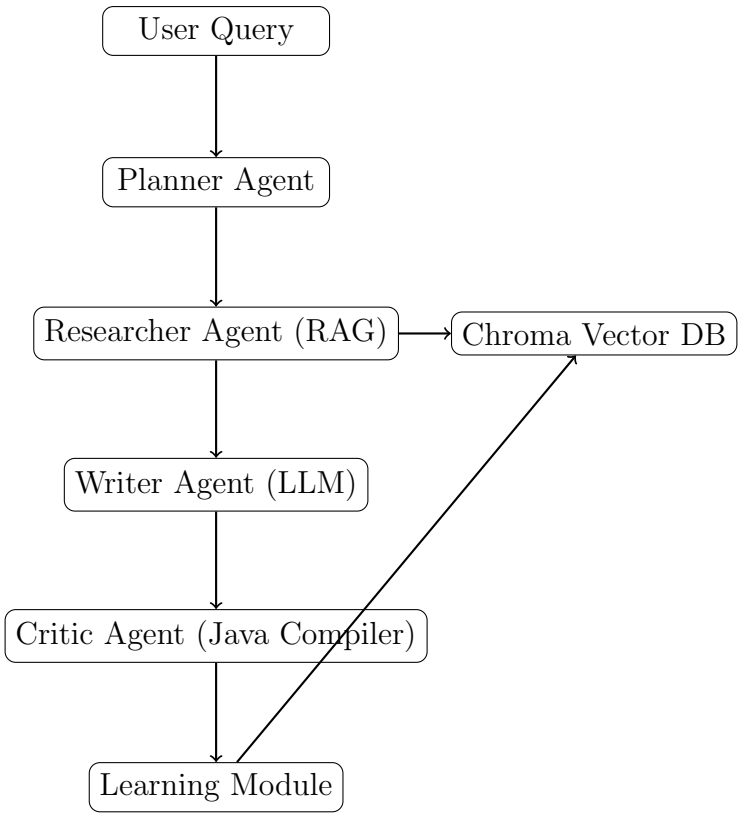
4.3 Grounded Generation

The language model generates code conditioned on the query and retrieved context.

$$\hat{s} = G_\theta(q, \mathcal{R}_k(q))$$

5 Agentic System Architecture

The system is implemented using **LangGraph**, which orchestrates multiple specialized agents.



6 Agent Responsibilities

6.1 Planner Agent

The planner converts a user request into structured reasoning steps.

6.2 Researcher Agent

The researcher performs semantic retrieval from the vector database using embeddings.

6.3 Writer Agent

The writer agent uses a language model to generate Java code based on:

- the user query
- the generated plan
- retrieved examples

6.4 Critic Agent

The critic validates generated code by compiling it with the Java compiler:

$$javac(code)$$

If compilation fails, the system retries generation.

7 Execution-Based Validation

Generated code is compiled using a sandboxed execution engine.

$$\mathcal{V}(s) = \begin{cases} 1 & \text{if compilation succeeds} \\ 0 & \text{otherwise} \end{cases}$$

This ensures that only syntactically valid Java programs are returned.

8 Memory and Continual Learning

The system includes two learning mechanisms:

8.1 Conversation Memory

Recent interactions are stored and provided as context to improve generation quality.

8.2 Continual Learning

When new queries produce valid code, the system stores them in the vector database.

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup \{(q, s^*)\}$$

This allows the assistant to continuously improve over time.

9 Serving Layer

The system is exposed as a REST API using FastAPI.

```
@app.post("/generate")
def generate_code(request: QueryRequest):

    result = graph.invoke({
        "query": request.query
    })

    return {
        "code": result["generated_code"],
        "valid": result["valid"]
    }
```

10 Dockerized Deployment

The entire system is containerized using Docker with CUDA support.

```
docker build -t java-rag .
docker run --gpus all -p 8000:8000 java-rag
```

This enables reproducible deployment across machines.

11 Evaluation Metrics

The system can be evaluated using:

11.1 Compilation Success Rate

$$CSR = \frac{\text{Successful Compilations}}{\text{Total Generations}}$$

11.2 Retrieval Relevance

Measured by cosine similarity.

11.3 Latency

$$T_{total} = T_{retrieve} + T_{generate} + T_{validate}$$

12 Challenges

- Limited GPU memory
- Prompt design for reliable code generation
- Cleaning LLM outputs to extract valid Java code
- Secure execution of generated programs

13 Future Work

Future improvements may include:

- Cross-encoder reranking
- AST-aware code retrieval
- automatic unit test generation
- multi-language code generation

14 Conclusion

This work demonstrates a practical implementation of an Agentic RAG system for Java code generation.

By combining semantic retrieval, multi-agent reasoning, execution-based validation, and continual learning, the system evolves from a static code generator into an adaptive programming assistant capable of improving through interaction.

The architecture remains lightweight enough to run on consumer hardware while maintaining strong generation quality and extensibility.